



Instituto Superior de Engenharia

Politécnico de Coimbra

DEPARTAMENTO DE / DEPARTMENT OF
INFORMÁTICA E SISTEMAS

Suricata (DPI and Lua Scripting)

Relatório de Licenciatura em / in Engenharia Informática

Autor / Author

Artur Meireles de Sousa Falcão

2024130485

Gabriel Cardoso Rodrigues

2024133397

Rodrigo Trindade Prazeres

2023139373

Orientador

Luís Eduardo Faria dos Santos

Unidade Curricular

Segurança



INSTITUTO POLITÉCNICO
DE COIMBRA

INSTITUTO SUPERIOR
DE ENGENHARIA
DE COIMBRA

Coimbra, 8 de junho de 2026

RESUMO

Este projeto foca-se na implementação e otimização de um Sistema de Detecção de Intrusões (IDS) utilizando o motor Suricata. O objetivo principal consistiu em ultrapassar as limitações das assinaturas estáticas através da utilização de *Deep Packet Inspection* (DPI) e *Lua Scripting*. O laboratório, emulado em ambiente GNS3, incluiu a configuração de um sensor de rede para monitorização de tráfego espelhado. O documento detalha os desafios técnicos enfrentados em ambientes virtualizados como anomalias de *checksum* e restrições de *sandboxing* e culmina no desenvolvimento de um conjunto de módulos de deteção dinâmica, comprovando a eficácia do sistema contra ataques Web complexos e tentativas de exfiltração de dados.

Palavras-chave: Suricata, DPI, *Lua Scripting*, IDS, GNS3.

ÍNDICE

Resumo	i
Índice	ii
Índice de figuras	iii
Lista de abreviaturas	iv
1 Introdução	1
2 Enquadramento Teórico: DPI e Lua <i>Scripting</i>	2
3 Arquitetura da Solução e Topologia	3
3.1 Segmentação Lógica	3
3.2 Mecanismo de Interceção de Tráfego	4
4 Desafios de Implementação e <i>Troubleshooting</i>	5
4.1 Necessidade de Endereçamento IP na Interface de Monitorização	5
4.2 Critérios de Seleção da Versão do Motor	5
5 Desenvolvimento de Módulos de Detecção em Lua	6
5.1 DLP: Detecção de Exfiltração de Dados	6
5.2 Detecção de Comportamento: Força Bruta	6
5.3 Análise de Camada de Aplicação: <i>User-Agents</i> Suspeitos	7
5.4 Controlo de Acessos: Bloqueio de Executáveis	8
6 Validação e Análise de Resultados	9
6.1 Detecção de Exfiltração de Dados (DLP)	9
6.2 Detecção de Comportamento: Força Bruta	9
6.3 Análise de Camada de Aplicação: Ferramentas Ofensivas	10
6.4 Controlo de Acessos: Bloqueio de Executáveis	10
7 Conclusão	11
Referências bibliográficas	12

ÍNDICE DE FIGURAS

3.1	Topologia de Rede.	3
5.1	Código de dlp_cartao.lua.	6
5.2	Código de forca_bruta_contador.lua.	7
5.3	Código de detecao_agente.lua.	7
5.4	Código de bloq_exec.lua.	8
6.1	Log DLP.	9
6.2	Log Força Bruta.	10
6.3	Log Ferramentas Ofensivas.	10
6.4	Log Bloqueio de Executáveis.	10

LISTA DE ABREVIATURAS

DPI	<i>Deep Packet Inspection</i>
HTTP	<i>Hypertext Transfer Protocol</i>
IDS	<i>Intrusion Detection System</i>
IOU	<i>Internetwork Operating System on Unix</i>
IP	<i>Internet Protocol</i>
OSI	<i>Open Systems Interconnection</i>
SPAN	<i>Switched Port Analyzer</i>
TCP	<i>Transmission Control Protocol</i>
UDP	<i>User Datagram Protocol</i>
VM	<i>Virtual Machine</i>

1 INTRODUÇÃO

O paradigma atual de segurança de redes enfrenta um desafio crescente: a sofisticação das técnicas de evasão utilizadas por atacantes para contornar sistemas de detecção tradicionais. Assinaturas baseadas puramente em *pattern matching* revelam-se frequentemente ineficazes contra tráfego ofuscado ou ameaças que alteram a sua estrutura dinamicamente.

Este trabalho propõe uma arquitetura de monitorização avançada, explorando o motor Suricata não apenas como um validador de cabeçalhos de rede, mas como um mecanismo DPI. O grande diferencial do projeto assenta na extensão destas capacidades através da linguagem Lua, permitindo desenvolver lógicas de detecção com estado (*stateful*), heurísticas de pontuação e inspeção profunda de *payloads*, dotando o IDS de inteligência adaptativa face a ameaças modernas [1, 2].

2 ENQUADRAMENTO TEÓRICO: DPI E LUA SCRIPTING

Para uma monitorização eficaz, o Suricata atua na Camada 7 (Aplicação) do modelo OSI. As funcionalidades críticas que suportam este projeto são:

- **DPI:** Ao invés da tradicional inspeção de estado (*Stateful Packet Inspection*), o DPI identifica protocolos independentemente dos portos e reconstrói fluxos (*Stream Reassembly*) para validar o conteúdo integral dos *payloads*.
- **Extensibilidade com Lua:** A injeção de lógica de programação no motor de inspeção permite normalizar dados em tempo real e aplicar lógicas condicionais complexas que as assinaturas estáticas nativas não suportam.

3 ARQUITETURA DA SOLUÇÃO E TOPOLOGIA

A emulação do ambiente empresarial foi realizada na plataforma GNS3, visando o isolamento de redes e a análise de tráfego fidedigno.

3.1 Segmentação Lógica

Conforme ilustrado na Figura 3.1, a infraestrutura baseou-se em equipamentos de encaminhamento Cisco (IOU), segmentada por:

- **Zona Externa:** Atacante (VM Kali Linux), responsável pela injeção de *payloads* maliciosos.
- **Core de Inspeção:** Servidor Ubuntu com motor Suricata. Embora desenhado para operar em modo *Stealth* (sem IP na interface de captura), a necessidade de estabelecer fluxos TCP reais para os testes de DPI obrigou à atribuição de um IP estático, garantindo a operacionalidade da inspeção sem comprometer a topologia de monitorização.
- **Zona Interna:** Vítimas e serviços alvo. Simulou-se um serviço web na porta 80, essencial para criar fluxos TCP e transações HTTP reais, permitindo que o Suricata realizasse a inspeção profunda (DPI) necessária para detetar os ataques injetados.

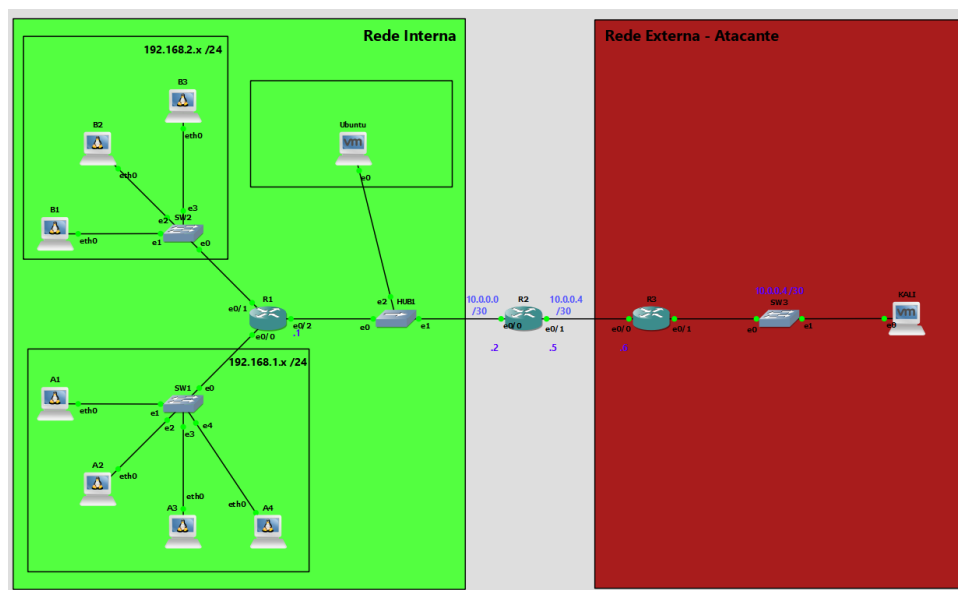


Figura 3.1: Topologia de Rede.

3.2 Mecanismo de Intercepção de Tráfego

Para a monitorização de tráfego, optou-se pela implementação de um *Hub* Ethernet posicionado no *backbone* da topologia. Esta abordagem tira partido da natureza de *broadcasting* nativa do equipamento, garantindo uma replicação física integral de todos os pacotes em trânsito para a interface do IDS. Esta solução de intercepção, além de assegurar uma fidelidade absoluta na replicação de dados para a análise de segurança, elimina qualquer sobrecarga de processamento nos *routers* de encaminhamento, preservando o desempenho dos equipamentos de rede durante a inspeção de tráfego.

4 DESAFIOS DE IMPLEMENTAÇÃO E *TROUBLESHOOTING*

A passagem de um cenário teórico para um ambiente virtualizado funcional exigiu a resolução de constrangimentos críticos:

4.1 Necessidade de Endereçamento IP na Interface de Monitorização

Durante a fase inicial de testes, verificou-se que o tráfego HTTP injetado pelo atacante (Kali Linux) não completava a transação com a vítima. Analisando os *logs* de rede, concluiu-se que o *stack* TCP do servidor Web não conseguia responder aos pedidos de conexão por falta de um endereço de rede na interface monitorizada pelo Suricata. A atribuição de um endereço IP (10.0.0.3/24) resolveu a anomalia, permitindo que o motor de DPI do Suricata processasse o tráfego de aplicação na totalidade sem interferir na topologia original de encaminhamento entre as restantes máquinas.

4.2 Critérios de Seleção da Versão do Motor

Inicialmente, ponderou-se a utilização da versão mais recente do motor Suricata (v8.0.3). Contudo, durante a fase de configuração da API Lua, verificou-se que a falta de exemplos práticos documentados na comunidade para a sintaxe desta nova versão dificultava o desenvolvimento célere dos módulos de deteção. Optou-se, portanto, por migrar para a versão 6.0.20, uma decisão fundamentada na vasta base de conhecimento disponível (*forums* e documentação oficial [1]), que permitiu uma implementação mais robusta e testada da lógica de DPI.

5 DESENVOLVIMENTO DE MÓDULOS DE DETECÇÃO EM LUA

Após a estabilização do motor DPI, desenvolveu-se um conjunto de scripts Lua focados em vetores críticos de ataque na Camada 7. Estes módulos permitem uma análise comportamental e semântica do tráfego, ultrapassando as limitações das regras estáticas do Suricata.

5.1 DLP: Detecção de Exfiltração de Dados

Objetivo: Proteção contra a exfiltração de dados sensíveis da rede interna. **Lógica:** O script `dlp_cartao.lua` (Figura 5.1) analisa o corpo das requisições HTTP (`http.request_body`) à procura de padrões numéricos que coincidam com números de cartões de crédito (formato XXXX-XXXX-XXXX-XXXX). **Aplicação:** Prevenção de fugas de informação confidencial em tráfego HTTP.

```
ubuntu@ubuntu:~$ cat /etc/suricata/lua/dlp_cartao.lua
function init (args)
    local needs = {}
    needs["http.request_body"] = tostring(true)
    return needs
end

function match(args)
    local body = tostring(args["http.request_body"])

    if body == nil or #body == 0 then
        return 0
    end

    if body:find("%d%d%d%d%-%d%d%d%d%-%d%d%d%d%-%d%d%d%d") then
        io.write("\nALERTA CRÍTICO: FUGA DE DADOS DETETADA!\n")
        io.write("Padrão de Cartão de Crédito encontrado no payload.\n")
        io.flush()
        return 1
    end

    return 0
end

ubuntu@ubuntu:~$ grep "sid:100001" /etc/suricata/rules/suricata.rules
alert http any any -> any any (msg:"[ALERTA LUA] Fuga de Cartao de Credito Detetada"; lua:/etc/suricata/lua/dlp_cartao.lua; sid:100001; rev:1;)
```

Figura 5.1: Código de `dlp_cartao.lua`.

5.2 Detecção de Comportamento: Força Bruta

Objetivo: Detecção de ataques de *Brute Force* a serviços de autenticação. **Lógica:** O script `forca_bruta_contador.lua` (Figura 5.2) implementa uma lógica *stateful* (com estado) que incrementa um contador global a cada tentativa de login falhada. Ao exceder o limiar de 3 tentativas, o sistema dispara um alerta crítico. **Aplicação:** Proteção contra tentativas de acesso não autorizado por adivinhação de credenciais (*dictionary attacks*).

```
ubuntu@ubuntu:~$ cat /etc/suricata/lua/forca_bruta_contador.lua
local contador = 0
local PASS_CORRETA = "cisco"

function init (args)
    local needs = {}
    needs["payload"] = tostring(true)
    return needs
end

function match(args)
    local p = args["payload"]
    if p == nil then return 0 end
    local texto = tostring(p)

    if string.find(texto, "user=admin") and string.find(texto, "pass=") then
        local pass_tentada = string.match(texto, "pass=([^\&]+)")

        if pass_tentada ~= PASS_CORRETA then
            contador = contador + 1
            io.write("\n[AVISO] Password incorreta! Tentativa #" .. contador .. "\n")

            if contador >= 3 then
                io.write("BLOQUEIO: 3 tentativas falhadas!\n")
                contador = 0
                io.flush()
                return 1
            end
        else
            contador = 0
            io.write("[INFO] Login bem sucedido. Contador resetado.\n")
        end
    end

    return 0
end

end

ubuntu@ubuntu:~$ grep "sid:100002" /etc/suricata/rules/suricata.rules
alert tcp any any -> any any (msg:"[ALERTA LUA] Ataque de Força Bruta Detetado"; lua:/etc/suricata/lua/forca_bruta_contador.lua; sid:100002; rev:1;)
```

Figura 5.2: Código de forca_bruta_contador.lua.

5.3 Análise de Camada de Aplicação: *User-Agents* Suspeitos

Objetivo: Identificação de ferramentas de ataque e *scanners* de vulnerabilidades. **Lógica:** O script *detecao_agente.lua* (Figura 5.3) inspeciona o cabeçalho *User-Agent* das requisições HTTP, filtrando assinaturas de ferramentas conhecidas como *sqlmap*, *nmap* ou *nikto*. **Aplicação:** Detecção proativa de fases de reconhecimento de um ataque informático.

```
ubuntu@ubuntu:~$ cat /etc/suricata/lua/detecao_agente.lua
function init (args)
    local needs = {}
    needs["payload"] = tostring(true)
    return needs
end

function match(args)
    local p = args["payload"]
    if p == nil then return 0 end

    local texto = tostring(p)

    if string.find(texto, "sqlmap") or string.find(texto, "nmap") or string.find(texto, "nikto") then
        io.write("\nFerramenta de scan detetada!\n")
        io.flush()
        return 1
    end

    return 0
end

end

ubuntu@ubuntu:~$ grep "sid:100003" /etc/suricata/rules/suricata.rules
alert tcp any any -> any any (msg:"[ALERTA LUA] Tool de Ataque Detetada"; lua:/etc/suricata/lua/detecao_agente.lua; sid:100003; rev:1;)
```

Figura 5.3: Código de detecao_agente.lua.

5.4 Controlo de Acessos: Bloqueio de Executáveis

Objetivo: Restrição de *downloads* maliciosos no perímetro da rede. **Lógica:** O script *bloq_exec.lua* (Figura 5.4) monitoriza pedidos HTTP (*GET*) à procura de extensões de ficheiros executáveis (*.exe*, *.sh*, *.bat*), que constituem vetores comuns para a instalação de *malware*. **Aplicação:** Aplicação de políticas de segurança no perímetro para impedir a entrada de executáveis não autorizados na rede interna.

```
ubuntu@ubuntu:~$ cat /etc/suricata/lua/bloq_exec.lua
function init (args)
    local needs = {}
    needs["payload"] = tostring(true)
    return needs
end

function match(args)
    local p = args["payload"]
    if p == nil then return 0 end

    local texto = tostring(p)

    if string.find(texto, "%.exe") or string.find(texto, "%.sh") or string.find(texto, "%.bat") then
        io.write("\nTentativa de download de executável bloqueada!\n")
        io.flush()
        return 1
    end
    return 0
end

ubuntu@ubuntu:~$ grep "sid:100004" /etc/suricata/rules/suricata.rules
alert tcp any any -> any any (msg:"[ALERTA LUA] Download Executavel Detetado"; lua:/etc/suricata/lua/bloq_exec.lua; sid:100004; rev:1;)
```

Figura 5.4: Código de *bloq_exec.lua*.

6 VALIDAÇÃO E ANÁLISE DE RESULTADOS

A validação foi efetuada através de ferramentas de auditoria ofensiva a partir da Zona Externa (Kali Linux). A análise aos registos (`fast.log`) confirmou a eficácia da DPI aliada à lógica Lua, com a geração de alertas precisos e contextualizados para cada vetor de ameaça.

6.1 Detecção de Exfiltração de Dados (DLP)

- **O que foi injetado:** Foi simulada uma tentativa de fuga de informação através do envio de um pedido HTTP POST contendo uma *string* numérica formatada como um cartão de crédito (1111-2222-3333-4444).
- **Como o Lua detetou:** O script `dlp_cartao.lua` intercetou o tráfego na Camada 7, extraiu o conteúdo de `http.request_body` e aplicou uma expressão regular (Regex) desenhada para identificar o padrão de 16 dígitos separados por hifens.
- **O alerta gerado:** O motor disparou um alerta de Prioridade 1 (Violação de Política), confirmando a quebra das regras de segurança de dados, como se pode observar na Figura 6.1.

```
ubuntu@ubuntu:~$ tail -f /var/log/suricata/fast.log
06/08/2026-01:37:44.863741  [**] [1:100001:1] [ALERTA LUA] Fuga de Cartao de Credito Detetada [**] [Classification: Potential Corporate Privacy Violation] [Priority: 1] {TCP} 10.0.1.2:45658 -> 10.0.0.3:80
```

Figura 6.1: Log DLP.

6.2 Detecção de Comportamento: Força Bruta

- **O que foi injetado:** O atacante realizou múltiplas submissões consecutivas de formulários de login com credenciais inválidas (parâmetro `pass=errada`).
- **Como o Lua detetou:** Ao contrário de uma regra estática, o script `forca_bruta_contador.lua` utilizou uma variável global para manter o estado (*stateful*) da ligação. O script contou as ocorrências em que a password extraída não correspondia à legítima. Ao atingir o limiar de três falhas, a condição crítica foi ativada.
- **O alerta gerado:** O registo refletiu a deteção exata do ataque de força bruta, sinalizando a anomalia comportamental, como se pode observar na Figura 6.2.

```
ubuntu@ubuntu:~$ tail -f /var/log/suricata/fast.log
06/08/2026-01:39:15.026403  [**] [1:100002:1] [ALERTA LUA] Ataque de Força Bruta Detetado [**] [Classification: Attempted Administrator Privilege Gain] [Priority: 1] {TCP} 10.0.1.2:44048 -> 10.0.0.3:80
```

Figura 6.2: Log Força Bruta.

6.3 Análise de Camada de Aplicação: Ferramentas Ofensivas

- **O que foi injetado:** Foi gerado tráfego web automatizado utilizando ferramentas clássicas de *scanning* e injeção de base de dados, embutindo os seus *footprints* padrão na requisição HTTP.
- **Como o Lua detetou:** O script `detecao_agente.lua` inspecionou integralmente o payload da requisição à procura de *strings* características destas ferramentas (ex: `sqlmap`, `nmap`, `nikto`).
- **O alerta gerado:** O sistema identificou o tráfego como gerado por uma ferramenta maliciosa e classificou-o adequadamente, emitindo o alerta no log, como se pode observar na Figura 6.3.

```
ubuntu@ubuntu:~$ tail -f /var/log/suricata/fast.log
06/08/2026-01:39:54.525730  [**] [1:100003:1] [ALERTA LUA] Tool de Ataque Detetada [**] [Classification: Web Application Attack] [Priority: 1] {TCP} 10.0.1.2:48388 -> 10.0.0.3:80
```

Figura 6.3: Log Ferramentas Ofensivas.

6.4 Controlo de Acessos: Bloqueio de Executáveis

- **O que foi injetado:** O utilizador simulou o descarregamento de um ficheiro com potencial malicioso, efetuando um pedido a um recurso terminado em `.exe`.
- **Como o Lua detetou:** O módulo `bloq_exec.lua` escrutinou o fluxo de texto do payload à procura de extensões críticas (`.exe`, `.sh`, `.bat`). Ao encontrar correspondência no padrão do URL requisitado, o pacote foi imediatamente sinalizado.
- **O alerta gerado:** Um alerta de violação de política foi gerado, documentando a tentativa de quebra do perímetro de segurança da rede interna, como se pode observar na Figura 6.4.

```
ubuntu@ubuntu:~$ tail -f /var/log/suricata/fast.log
06/08/2026-01:40:37.899381  [**] [1:100004:1] [ALERTA LUA] Download Executavel Detetado [**] [Classification: Potential Corporate Privacy Violation] [Priority: 1] {TCP} 10.0.1.2:52148 -> 10.0.0.3:80
```

Figura 6.4: Log Bloqueio de Executáveis.

7 CONCLUSÃO

O laboratório demonstrou as vantagens da transição de *firewalls* convencionais para Sistemas de Detecção de Intrusões programáveis. Apesar dos inúmeros desafios impostos por anomalias em ambientes virtuais, a emulação foi bem-sucedida. O Suricata, suportado pela agilidade do Lua *Scripting*, provou ser um sensor formidável, capaz de não apenas detetar, mas também pontuar, classificar e auditar profundamente fluxos de rede, consolidando a sua relevância arquitetónica na proteção de infraestruturas modernas.

REFERÊNCIAS BIBLIOGRÁFICAS

- [1] Open Information Security Foundation (OISF), *Suricata 6.0.20 Documentation*, 2026, documentação oficial da versão utilizada no laboratório. [Online]. Available: <https://docs.suricata.io/en/suricata-6.0.20/index.html>
- [2] R. Ierusalimschy, L. H. de Figueiredo, and W. Celes, *Lua 5.1 Reference Manual*, 2006. [Online]. Available: <https://www.lua.org/manual/5.1/>



**Instituto Superior
de Engenharia**

Politécnico de Coimbra